

Exercise 5: Individual Mini-Capstone (Live) — 35 minutes

Scenario

Pick **one** task below. Run the full loop — restate, plan, implement, verify, review — without anyone prompting you through the steps. This exercise has no instructor-led walkthrough; it's the four practical exercises' method, applied unaided.

Each task sheet below gives you everything the deck promises: a short written requirement, acceptance tests, a list of allowed files, a suggested test command, and one deliberate ambiguity left for you to resolve — not an oversight, a decision OpenCode should surface and you should make.

Choose one task

Task 5.A — Add a validation rule for an invalid quantity

Requirement: `InventoryService.validateForCreate` currently rejects a negative `quantity` but has no upper bound. Add a maximum: a `quantity` greater than `100000` must be rejected on create, the same way a negative one is.

Acceptance tests:

- `quantity = 100000` is accepted (boundary is inclusive).
- `quantity = 100001` is rejected with a clear `InventoryException` message.
- `quantity = -1` is still rejected exactly as it is today (don't regress the existing rule).
- `quantity = 0` is still accepted.

Allowed files: `src/main/java/com/atlas/inventory/InventoryService.java` and either `src/test/java/com/atlas/inventory/InventoryServerTest.java` (extend it) or a new `InventoryServiceTest.java`.

Suggested test command: `mvn test -Dtest=InventoryServerTest` (or `InventoryServiceTest`, whichever you land on).

Deliberate ambiguity (two of them here):

- The task sheet does not say whether `reorderLevel` should have the same upper bound. Decide, and be ready to explain why.

- Look at `validateForUpdate` before you write anything. It does not check `quantity` at all — so should your new maximum apply to updates too? Whichever way you answer, say so explicitly rather than letting the agent decide for you by accident.

Task 5.B — Add a stock-status category with examples

Requirement: Add a derived stock status to `InventoryItem`, similar in spirit to a pricing tier: `OUT_OF_STOCK` when `quantity == 0`, `LOW` when `quantity` is above `0` but at or below `reorderLevel`, `OK` otherwise.

Acceptance tests:

- `quantity = 0, reorderLevel = 5` → `OUT_OF_STOCK`.
- `quantity = 3, reorderLevel = 5` → `LOW`.
- `quantity = 5, reorderLevel = 5` → `LOW` (at the threshold, not yet `OK`).
- `quantity = 6, reorderLevel = 5` → `OK`.

Allowed files: `src/main/java/com/atlas/inventory/InventoryItem.java` and a new or extended `InventoryItemTest.java`.

Suggested test command: `mvn test -Dtest=InventoryItemTest`.

Edge-case check: Because the rule above makes `quantity = 0` `OUT_OF_STOCK` regardless of `reorderLevel`, add a test for `quantity = 0, reorderLevel = 0` expecting `OUT_OF_STOCK`.

Task 5.C — Fix a boundary error at zero

Trainer setup (apply before picking this task): add this method to `InventoryItem.java`:

```
public int stockLevelPercentage() {  
    return (quantity() * 100) / reorderLevel();  
}
```

and this failing test to `src/test/java/com/atlas/inventory/InventoryItemTest.java` (create that file if you didn't do exercise 2 — copy the package declaration and imports from `InventoryServiceTest.java`):

```

@Test
void stockLevelPercentageHandlesNoReorderLevel() {
    InventoryItem item = new InventoryItem(
        0, "A-1", "No Reorder Tracking", "Component", "A-01-01", 12, 0, "");

    assertEquals(100, item.stockLevelPercentage());
}

```

Confirm it currently throws `ArithmeticException: / by zero` rather than returning `100`.

Requirement: fix `stockLevelPercentage()` so an item with `reorderLevel = 0` (meaning: this item isn't tracked for reordering at all) is treated as fully stocked — it should return `100`, not divide by zero.

Acceptance tests:

- `reorderLevel = 0` → `100`, regardless of `quantity` (as long as `quantity >= 0`).
- `quantity = 5, reorderLevel = 10` → `50`.
- `quantity = 10, reorderLevel = 10` → `100`.
- `quantity = 20, reorderLevel = 10` → `200` (still allowed to exceed 100 — this method reports a ratio, not a capped percentage).

Allowed files: `src/main/java/com/atlas/inventory/InventoryItem.java` and `InventoryItemTest.java` only.

Suggested test command: `mvn test -Dtest=InventoryItemTest`.

Deliberate ambiguity: the task sheet doesn't say whether the fix belongs as a guard clause at the top of the method or as part of the arithmetic itself. Either is defensible — decide, and say why in your review.

Task 5.D — Add a focused test for an unhandled edge case

Requirement: `InventoryService.validateTextFields` rejects a `description` longer than 500 characters, but nothing in the test suite confirms the boundary itself — that exactly 500 characters is accepted and 501 is not.

Acceptance tests (these are the tests you're adding, not code you're changing):

- A description of exactly 500 characters is accepted.
- A description of 501 characters is rejected with the existing message.
- An empty description is still accepted (it's optional).

Allowed files: a new `InventoryServiceTest.java`, or an addition to `InventoryServerTest.java` — **no production code should change for this task.**

Suggested test command: `mvn test -Dtest=InventoryServiceTest` (or `InventoryServerTest`).

Deliberate ambiguity: the task sheet doesn't say which layer to test at — directly against `InventoryService`, or through the HTTP API via `InventoryServerTest`. Decide, and be ready to explain the tradeoff.

Task 5.E — Improve an error message without changing the API

Requirement: when a duplicate part number is submitted, `InventoryService` currently reports: "An inventory item with that part number already exists". It doesn't say *which* part number. Improve the message to include the actual value, without changing `InventoryException`'s type, its `ErrorType`, or the HTTP status code it maps to (`409`).

Acceptance tests:

- Submitting a duplicate `partNumber` of "RES-10K" produces a message that includes `RES-10K`.
- The exception is still `InventoryException` with `ErrorType.CONFLICT`.
- The existing test `InventoryServerTest#reportsDuplicatePartNumbersAsConflict` still passes — check what substring it currently asserts on before you change the wording, so you don't break it by accident.

Allowed files: `src/main/java/com/atlas/inventory/InventoryService.java` only. Do not touch the test unless you're certain the existing assertion is compatible with your new wording — improving the message is not license to weaken the test.

Suggested test command: `mvn test -Dtest=InventoryServerTest`.

Deliberate ambiguity: the task sheet doesn't specify exact wording or whether the part number should be quoted. Decide on a phrasing, and check it against `AGENTS.md` if your repository uses one and it has an opinion on message style.

Hint, because this one has real teeth: read `InventoryServerTest#reportsDuplicatePartNumbersAsConflict` first. The wording you pick has to satisfy both the new requirement and that existing assertion, and most natural rewrites fail one of them. Work out what the constraint actually is before you let the agent start writing.

Prompt template

Fill in the brackets for whichever task you picked. Keep the structure.

You are helping with a small, well-bounded change.

Task: [insert the requirement from your chosen task sheet]

Acceptance criteria:

- [criterion 1]
- [criterion 2]
- [criterion 3]

Constraints:

- Inspect only the relevant class(es) named below.
- Modify only these files: [file list from your task sheet's "Allowed files"].
- Do not add dependencies.
- Preserve existing public APIs.
- Run [test command from your task sheet] after editing.
- Stop and ask if the requirement is ambiguous.

Start by restating the task and giving a plan in five steps or fewer. Wait for my approval before editing.

A note on tests

Tasks 5.A and 5.D need a test that exercises `InventoryService` directly. `src/test/java/com/atlas/inventory/InventoryServiceTest.java` already has the wiring for that — a `@TempDir` SQLite file, a repository, and a service — so extend it rather than building a harness from scratch. Do not let the agent reach for an in-memory database; this repository opens a fresh connection per operation, so an in-memory one is empty again by the time the query runs.

Hand back six things

When you're done, write up — for yourself, or for whoever is reviewing your capstone:

1. **A restatement of the task**, in your own words, not OpenCode's.
2. **The plan** OpenCode proposed, as you approved it (or as you asked it to revise).
3. **The implementation** — what actually changed.
4. **Tests or verification evidence** — the exact command you ran and its output.
5. **A final diff review** — `git diff`, read line by line.

6. One paragraph on where you overruled the agent.

Item 6 is the point of the exercise. If you never overruled it — never pushed back on the plan, never rejected part of the diff, never resolved the deliberate ambiguity differently than OpenCode assumed — you were not supervising, you were watching.

Debrief

Would you approve your own capstone's diff without having read it? If the honest answer is no, what specifically would you still want to check before you did?